HOCSS: A Hardware-accelerated Optical Circuit Switch Scheduler for low-latency Optimal Ports Matching

Yangmiao Yu[1,2], Yuexiang Song[1,2], Yifan Zhang[1,2], Dawei Zang[1,2,3*]

[1] *Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China*
[2] *University of Chinese Academy of Sciences*
[3] *State Key Lab of Processors, ICT, CAS*

[*] *Correspinding author: zangdawei@ncic.ac.cn*
All authors contributed equally to this manuscript.

Abstract: We propose a low-latency optimal ports matching scheduler for optical circuit switches. Leveraging the hardware-accelerated computation architecture, in optimal cases, port matching latency is less than 1us, achieving up to a 68.8× speedup compared to CPU implementation.

1. Introduction

Emerging artificial intelligence(AI) applications such as large language models are significantly increasing data traffic within data center networks and high performance computing systems. Unlike traditional electronic packet switching, which introduces delays due to processing overhead, optical circuit switch (OCS) enables dedicated optical paths for data transmission with high bandwidth and lower latency, which is crucial for applications requiring real-time data processing, such as high-frequency trading, large-scale cloud computing, and AI workloads. These advantages have led to widespread adoption in hyper scale data centers, with companies such as Google substantially increasing their deployment of OCS infrastructure to support their AI computing workloads.[1]

However, the performance of optical switch fabrics is often bottlenecked by the latency of the schedulers. In the scenarios, a critical challenge in managing the OCS optical paths is efficiently matching input and output ports of the optical switch to optimize the total data throughput. Usually, optimal port matching is formulated as a maximum bipartite matching problem and solved in polynomial time by applying the optimization algorithm, such as Hungarian Algorithm. Due to the high complexity of the optimization algorithm, the time required to complete an optimal matching calculation can take several tens of milliseconds which is the same order of the switching time of current OCS devices. Limited by the switching and scheduling time, the optical switches can only handle network flow with a large amount of data, called elephant network flow, with low bandwidth utilization. With the advancement of photonic integration technology, optical switches based on integrated optical waveguides can provide fast switching ranging from ns to 10μs[2], in contrast to the traditional devices. However, the computational time of optimal ports matching algorithm lags significantly behind for the reason of the limitations of CPU performance. The order-of-magnitude gap between scheduling and switching time results in substantial bandwidth under utilization and increases end-to-end packet latency, limiting the adaptability of OCS device to a broader range of applications.

In the paper, we propose HOCSS, a hardware-accelerated optical circuit switch scheduler for low-latency optimal ports matching. In the scheduler, a novel parallel architecture corresponding to computational features of the Hungarian Algorithm is designed to reduce the overall ports matching calculation time. We implement a HOCSS prototype on the AMD V80 FPGA board and evaluate the performance of various port size switches from 64 to 1024. The results show that the proposed scheduler can reduce the computational time to the microsecond order, achieving up to a 68.8× speedup in optimal cases compared to traditional CPU implementations.

2. HOCSS Architecture

To effectively utilize OCS networks, the servers or datacenter management systems send link requests, including input port, output port, and traffic size information, to OCS scheduler in a certain time interval. As illustrated in Figure 1, based on the link requests, the scheduler forms a traffic matrix to present the bandwidth requirements between each input ports and output ports. In order to maximize the bandwidth utilization of the OCS links as much as possible, the ports matching tasks is modelled as a maximum bipartite matching problem. A matching in a bipartite graph is a set of the edges chosen in such a way that no two edges share an endpoint, and a maximum matching is a matching of maximum the weight sum of selected edges. In the OCS link schedule scenario, the optimization objective involves maximizing aggregate bandwidth utilization based on traffic matrix constrained

by conditions that no two links share a same input port or output port. Hungarian algorithm has emerged as the predominant solution due to its ability to find optimal matchings that maximize total traffic in polynomial time.[3] This work adopts a matrix-based Hungarian algorithm, which converts the maximum traffic matrix to a minimum traffic matrix. The algorithm performs row and column subtraction operations on the traffic matrix, iteratively marks zero elements, searches for augmenting paths, and adjusts row and column coverings to ultimately find the optimal matching. During the Hungarian algorithm iteration process, the key steps involve finding minimum values, searching for augmenting paths, and determining step transitions based on the status of zero elements.

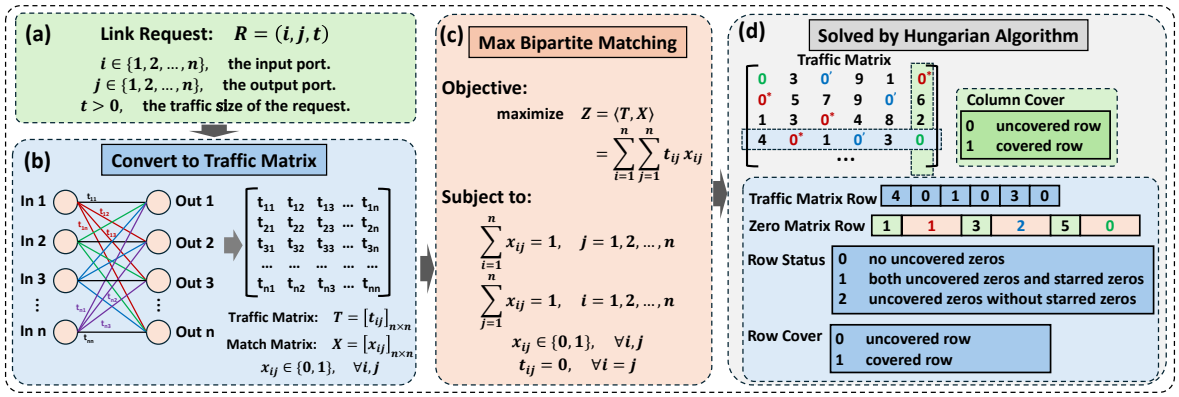


Fig. 1: Optimized Port Matching Process

To adapt to the HOCSS architecture, this work optimizes the implementation of the Hungarian algorithm. As illustrated in Figure 1(a), each row of the traffic matrix resides in a single URAM block. Each row of the traffic matrix corresponds to a row in the zero matrix, with each zero matrix row also stored in a single URAM block. The zero matrix rows maintain the positions and statuses of zero elements from the corresponding traffic matrix rows, where starred zeros are marked as 1, primed zeros as 2, and unmarked zeros as 0. When searching for zero elements, HOCSS scans the corresponding zero matrix rows within each PE. Row status values track the state of all zero elements in each traffic matrix row: status 0 indicates no uncovered zeros, status 1 indicates both uncovered and starred zeros present, and status 2 indicates uncovered zeros without starred zeros. By comparing row status values across all rows, HOCSS can rapidly determine whether the next step is to search for augmenting paths or to find minimum values and update the traffic matrix. Based on the row status values, HOCSS can also efficiently locate the rows containing the zero elements required for subsequent steps.

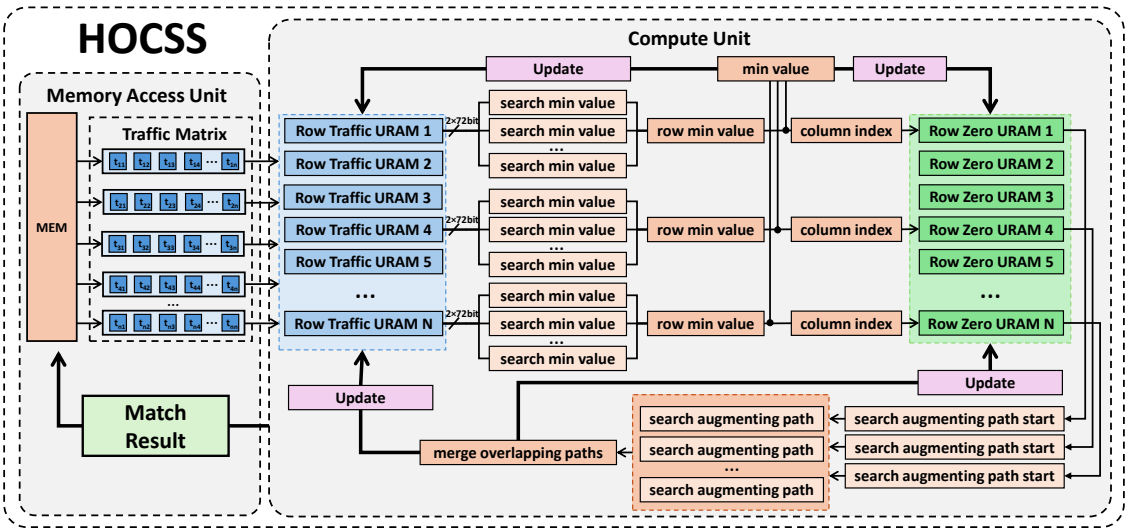


Fig. 2: HOCSS Architecture

As illustrated in Figure 2, the HOCSS architecture consists of two primary components: the memory access unit and the compute unit. The memory access unit is responsible for loading the traffic matrix from memory to Ultra RAM (URAM) and writing back the matching results to memory upon completion. Both the traffic matrix and intermediate computational results are stored in on-chip URAM blocks to minimize memory access latency. The compute unit comprises multiple processing elements (PEs) that execute various steps of the Hungarian algorithm,

with each PE maintaining independent URAM for one or more rows of the traffic matrix.

To minimize Hungarian algorithm execution latency, HOCSS accelerates the algorithm's key steps. As illustrated in Figure 2, the architecture utilizes dual-port URAM with 72-bit bandwidth per port, enabling each PE to read and compare multiple traffic matrix elements in parallel within a single clock cycle, thereby accelerating the minimum value search. The column index of the row minimum value searched is subsequently written as element to the Zero URAM. The minimum value search hardware structure used for constructing the Zero Matrix is reused for subsequently finding minimum values and updating the traffic matrix. HOCSS employs parallel search strategies to accelerate augmenting paths searching. Through row status values, HOCSS identifies rows containing the starting zero elements for augmenting paths. By accessing the corresponding Zero URAM for these rows, HOCSS locates the augmenting path starting points. HOCSS starts parallel searches from multiple starting points simultaneously, and upon completion of the search phase, merges the overlapping paths that were discovered during the search.

3. Experimental Setup and Results

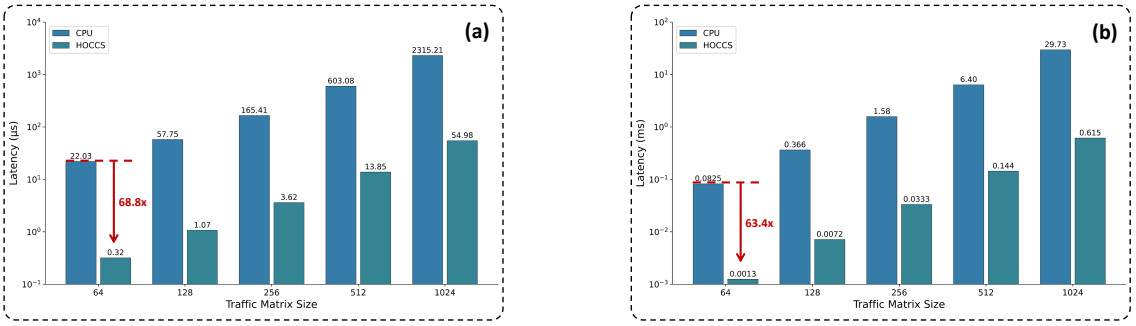


Fig. 3: Scheduling Latency Under Random Traffic Scenarios

To evaluate the performance of the proposed architecture, we implement a hardware prototype on AMD V80 FPGA board[4]. The design is synthesized and implemented using the Xilinx Vitis 2024.1 and Vivado 2024.1 toolchains, with an operating frequency set at 300 MHz. As a baseline for comparison, we perform the Hungarian algorithm provided by the Python SciPy library on a CPU platform equipped with an AMD Ryzen 9950X processor. We compare the computational latency of various switch radix sizes ranging from 64 to 1024 ports.

Figure 3(a) shows the best latency of many random tests, and HOCSS demonstrates exceptional low-latency characteristics. For the switch radix of 64×64, the best latency of HOCSS is a mere 0.32μs, compared to 22.03μs of the CPU implementation, yielding a speedup of 68.8×. For the switch radix of 1024×1024, HOCSS completes the computation in 54.98μs, drastically outperforming the CPU. Figure 3(b) show the average latency of random tests, HOCSS maintains a significant performance advantage. For the size of 256×256, the CPU latency reaches the millisecond level while HOCSS latency remains at the 10us level; for the size of 1024×1024, the CPU latency reaches 29.73ms while HOCSS latency remains at the microsecond level. In summary, the experimental results confirm that HOCSS consistently reduces the scheduling computation latency to the microsecond level across different traffic patterns. Compared to traditional CPU implementation, it achieves up to 68.8× performance improvement. These results highlight the potential of HOCSS to effectively bridge the gap between optical circuit switch and scheduler.

4. Conclusion

In this work, we design a hardware-accelerated optical circuit switch scheduler for low-latency optimal ports matching and implement a FPGA prototype. Experimental results demonstrate that it significantly reduces the latency by 68.8× and adapts to the faster switching devices .

References

[1] Google OCP. The open compute project announces new optical circuit switching (ocs) project.

[2] et al. Konoike. Path-independent insertion loss 8× 8 silicon photonics switch with nanosecond-order switching time. *Journal of Lightwave Technology*, 41(3):865–870, 2023.

[3] et al. Munkres. Algorithms for the assignment and transportation problems. *Journal of the society for industrial and applied mathematics*, 5(1):32–38, 1957.

[4] AMD. Amd v80 fpga board datasheet summary.

Optical Fiber Communication Conference and Exhibition 2026

HOCSS: A Hardware-accelerated Optical Circuit Switch Scheduler for low-latency Optimal Ports Matching

Yangmiao Yu[1,2], Yuexiang Song[1,2], Yifan Zhang[1], Dawei Zang[1]
[1]Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China
[2]University of Chinese Academy of Sciences, Beijing, China

Abstract

We propose a low-latency optimal ports matching scheduler for optical circuit switches(OCS). Leveraging the hardware-accelerated computation architecture, in optimal cases, port matching latency is less than 1us, achieving up to a 68.8× speedup compared to CPU implementation.

Introduction

- Emerging AI applications are driving data traffic growth. OCS enables high-bandwidth, low-latency optical paths, leading to widespread adoption in data centers.
- Optimal ports matching takes tens of milliseconds, bottlenecking OCS performance. While optical waveguides provide switching from ns to 10μs, scheduling latency lags far behind.
- We propose HOCSS, a hardware-accelerated OCS scheduler, reducing ports matching latency to microsecond order, achieving up to 68.8× speedup.

Optimized Ports Matching

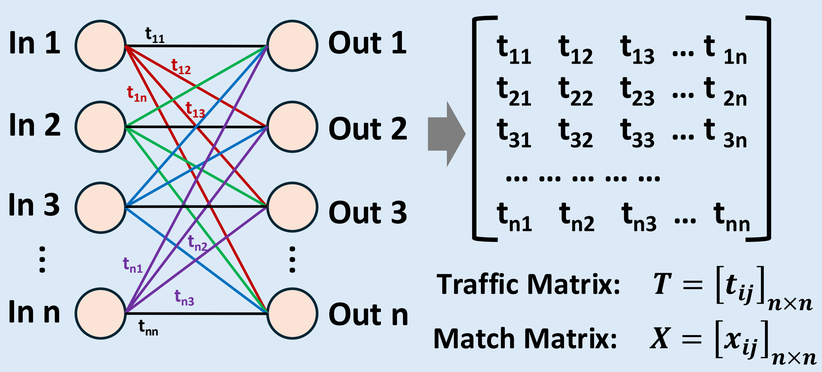
- The ports matching task is modelled as a maximum bipartite matching problem, maximizing aggregate bandwidth utilization constrained by conditions that no two links share a same input port or output port.

Link Request

Link Request: $R = (i, j, t)$

$i \in \{1, 2, \dots, n\}$, the input port.
$j \in \{1, 2, \dots, n\}$, the output port.
$t > 0$, the traffic size of the request.

Convert to Traffic Matrix



Traffic Matrix: $T = [t_{ij}]_{n \times n}$
Match Matrix: $X = [x_{ij}]_{n \times n}$
$x_{ij} \in \{0, 1\}, \quad \forall i, j$

Max Bipartite Matching

Objective:

$$maximize \quad Z = \langle T, X \rangle = \sum_{i=1}^{n} \sum_{j=1}^{n} t_{ij}\, x_{ij}$$

Subject to:

$$\sum_{i=1}^{n} x_{ij} = 1, \quad j = 1, 2, \dots, n, \qquad \sum_{j=1}^{n} x_{ij} = 1, \quad i = 1, 2, \dots, n$$

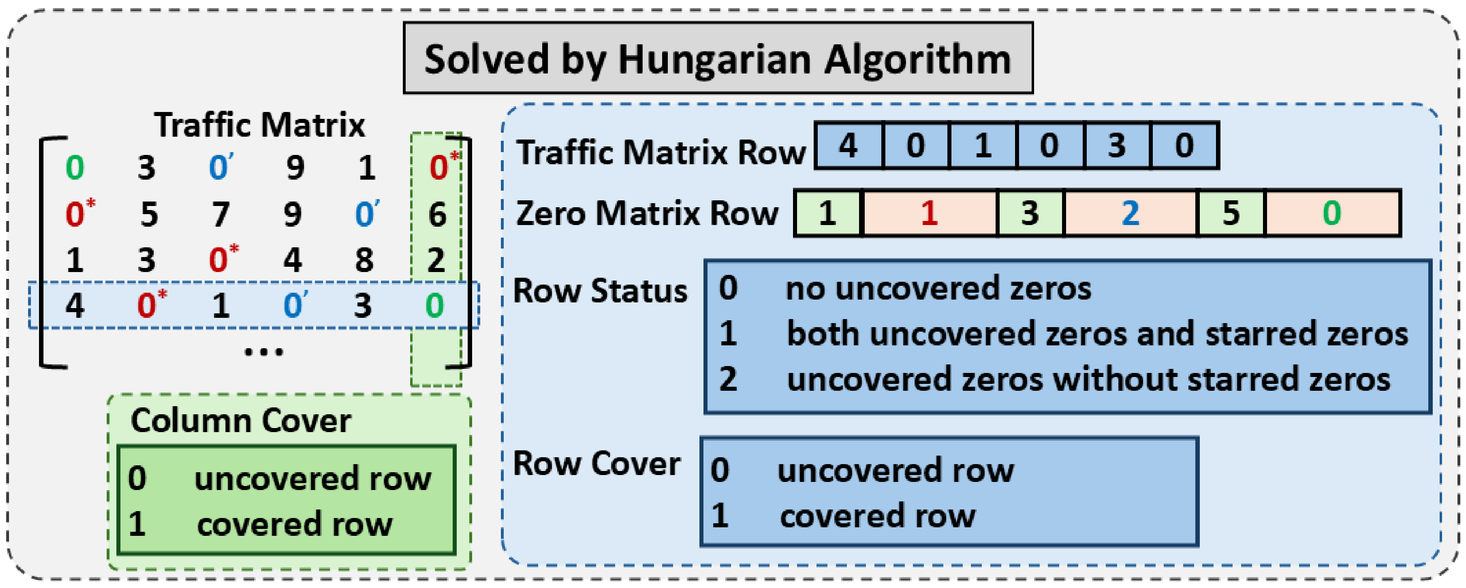
$$x_{ij} \in \{0, 1\}, \quad \forall i, j, \qquad t_{ij} = 0, \quad \forall i = j$$

Conclusion

In this work, we design a hardware-accelerated optical circuit switch scheduler for low-latency optimal ports matching and implement a FPGA prototype. Experimental results demonstrate that it significantly reduces the latency by 68.8× and adapts to the faster switching devices .

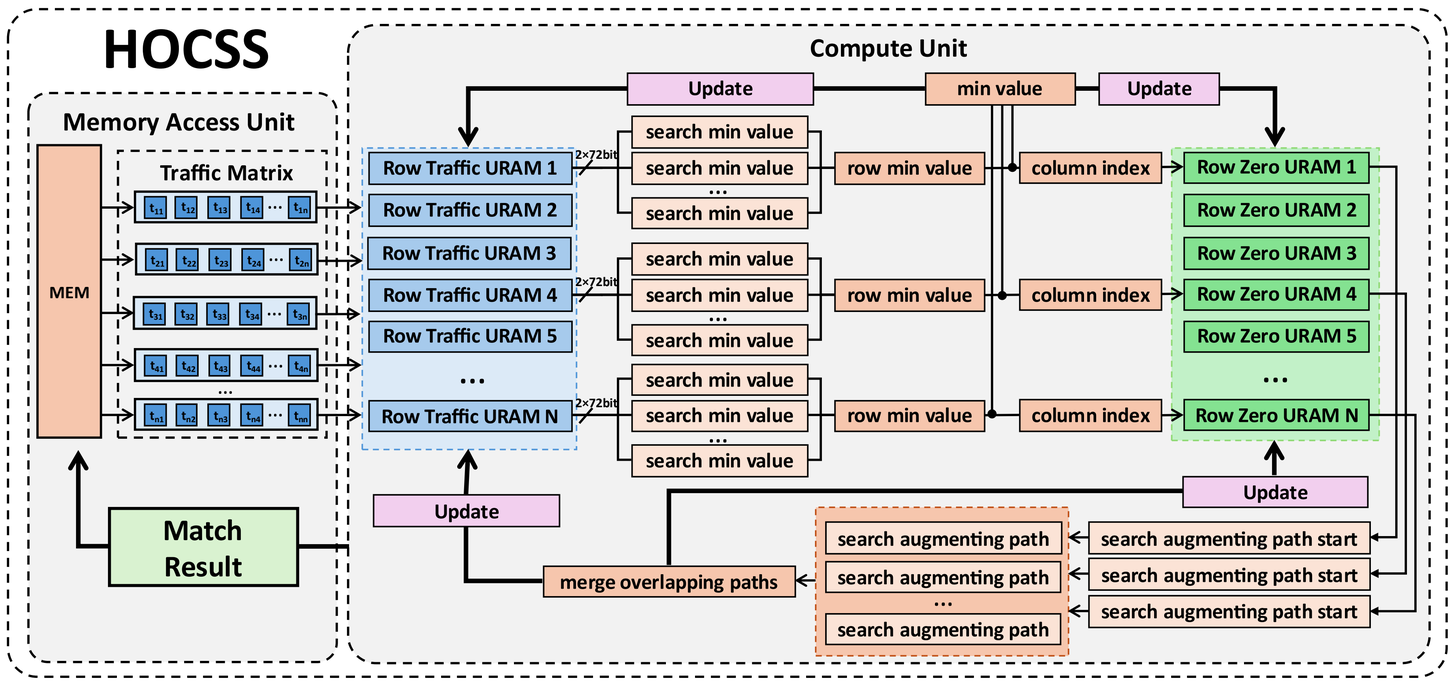
Hungarian Algorithm

- The matrix-based Hungarian algorithm is adopted to find optimal matchings in polynomial time. Each row of the traffic matrix resides in a single URAM block, and by comparing row status values, HOCSS rapidly determines whether the next step is augmenting path search or minimum value update.



HOCSS Architecture

- HOCSS architecture consists of the memory access unit and the compute unit. Both the traffic matrix and intermediate results are stored in on-chip URAM to minimize memory access latency.
- Dual-port URAM with 72-bit bandwidth enables each PE to read and compare multiple elements in parallel. Through row status values, HOCSS identifies starting points for augmenting paths and starts parallel searches simultaneously, merging overlapping paths upon completion.



Result

- Across many random instances, for 64×64, HOCSS achieves a best-case latency of 0.32 μs versus 22.03 μs on the CPU, a 68.8× speedup. For 1024×1024, the CPU's average latency reaches 29.73 ms, while HOCSS remains in the microsecond level.